Note on Single-Source Shortest Path Problem *

Zhongkui Ma

February 19, 2022

1 Instruction

When physically suspending a net, all the stretched edges are part of the shortest path, and those slack edges do not belong to the shortest path because shorter paths will be tightened earlier, while longer paths will become slack. This process is similar to BFS, but BFS cannot distinguish whether the edge is taut or loose. If there is no cycles in a graph, then BFS will find all the shortest paths in the undirected graph; when there is one cycle in a graph, BFS may find the wrong path. The most famous single-source shortest path algorithm is proposed by Dijkstra. It selects the vertex with the shortest distance currently in the traversal of each vertex, which is a correct and optimal choice, but it is unnecessary. It also pays the price of not handling graphs with negative weights. This paper designs two strategies, backtracking and absorption, to solve the shortest path problem without using this step. This process also shows the essential difference between the shortest path problem and the BFS.

2 Algorithm

2.1 Algorithm Description

Here, the graph G defaultly is an undirected simple graph.

Definition 1 *The **shortest-path tree** $T(i) = (V(T), E(T))$ means that, in the graph $G = (V(G), E(G))$, a vertex $i \in V(G)$ as the starting vertex, the union of the shortest paths $\{S(i,j)\}$ to all other vertices $j \in V$ obtained by the shortest path algorithm; that is, $T(i) = \cup_{j \in V} S(i,j)$.*

There are some definitions in algorithms. A **stem** refers to a vertex whose all adjacent vertices have been traversed. A **leaf** refers to a vertex whose at least one adjacent vertex (except for the parent vertex) has not been traversed. A vertex that is neither a stem nor a leaf is a vertex that has not been traversed. The **buds** of one vertex refers the number of adjacent vertices that are not stems. The distance $s(i)$ of a vertex i refers to the path's length from the vertex to the root vertex of the shortest path tree.

The following describes an algorithm for generating the shortest path tree and it is a single-source shortest path algorithm (see an example in Figure 1).

1. Initialize all parameters. For each vertex i: set their distance $s(i)$ to infinity and parent and child vertices to be null; set them to be neither leave vertex nor stem; set the number of their respective buds $b(i)$ equal to the degree of connectivity. For the graph G, if the edge (i,j) does not exist, set its weight to infinite. For an undirected graph, the edge (i,j) and the edge (j,i) are the same one.

2. Choose a starting vertex r, set $s(r) = 0$, and set it as a leaf. Enqueue it in the queue.

3. When the queue is not empty, dequeue a vertex and m. If m is a leaf, traverse each adjacent vertex n and the following strategies are executed:

   - n is neither a leaf nor a stem: Set the parent vertex of n to m, let n be a leaf and enqueue it. And set n as a child vertex of m.
   - (**Connection**) n is a leaf: Mark m as p and n as c and perform the following operations (Figure 2):

*This note does not guarantee the correctness and you can email to me (*zhongkui.ma@uqconnect.edu.au*) if you are interested or have problems with it.*

(a) (**Backtracking**) Let w_{pc} be the weight of edge (p, c). Calculate the distance $d_{S'}(c)$ along the path branch S' where c is located and the distance $d_S(p)$ along the path branch S where p is located. If the distance of the vertices in S' or S has calculated, there is no need to recalculate. If $d_S(p) + w_{pc} \geq d_{S'}(c)$, exit the loop.

(b) (**Absorption**) The distance of c to update is $d_S(p) + w_{pc}$ and set the parent vertex of c to be the child vertex of c and c to be the child vertex of p. If the distance of descendants of c is not infinity, reset to infinity.

The number of buds of m and n is reduced by one. If $b(n)$ is 0, set n not as a leaf but as a stem. Perform the same procedure with m. Mark the original parent vertex of c and c as p and c respectively, and repeat the above connection steps.

4. Select the root vertex r, and traverse all descendants to calculate the distance.

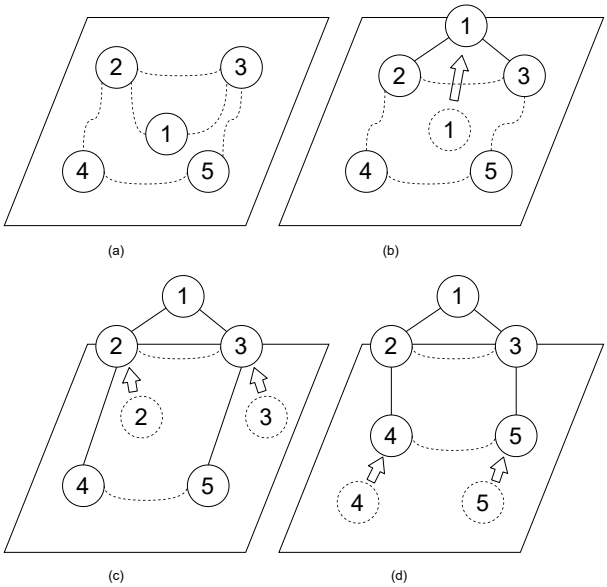

Figure 1: A Suspending Process (The figure shows a processing of suspending a network. According to the distances of each vertex to the root vertex, the other vertices will be suspended successively. The dash edges are slack edges and the solid edges are tighten edges.)

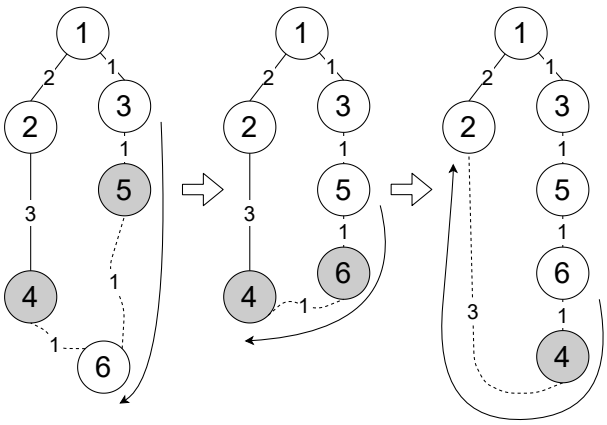

Figure 2: A Connection Process (The gray vertices are leaves and the white vertices are stems. The dash edges are slack edges and the solid edges are tighten edges.)

2.2 Algorithm Complexity

Assuming the graph G has $|E|$ edges and $|V|$ vertices using adjacency mapping to store the graph. The time complexity analysis of steps 1, 2 and 4 is simple. The time complexity of step 1 is $O(|E|+|V|)$. All vertices and connections are traversed and initial parameters are set. The time complexity of step 2 is $O(1)$. Only the parameter value of the starting vertex is set. The time complexity of step 4 is $O(|E|+|V|)$. This step recalculates the distances of all vertices. It only needs to traverse all the vertices and edges of the tree T.

Before explaining the time complexity of step 3, first, describe the following theorems.

Theorem 1 *A path $S = (V(S), E(S))$, where $V(S) = \{v_1, v_2, \cdots, v_n\}$, $E(S) = \{(v_1, v_2), (v_2, v_3), \cdots, (v_{n-1}, v_n)\})$ is in the graph G. If S is backtracked N $(N > 1)$ times in the algorithm, the directions of two adjacent backtracking are not the same.*

Proof *Suppose the current shortest path tree generated in the algorithm is $T(r)$. Let the parent vertex of the current v_1 be v_{q0}. Suppose the sum of weights of all edges composing the path S is $w_S = \sum_{i=1}^{n-1} w(v_i, v_{i+1})$. The new vertices that cause N backtracking are $\{v_{qi} | 0 < i \leq N\}$.*

There is another vertex $v_{q1} \notin V(S)$ connected with v_n and set $d_T(v_1) = l_1$. If $d_T(v_{q1}) + w_S < l_1$, the path S is traced back and absorbed once. After that, v_{q1} is the parent vertex of v_n and v_{i+1} is the parent vertex of v_i $(n > i \geq 1)$.

Therefore, if the path S is backtracked twice, it must be another vertex $v_{q2} \notin V(S)$ connected with v_1, and set $d_T(v_n) = l_2$, then $d_T(v_{q2}) + w_S < l_2$ and the path S is backtracked and absorbed another time. After that, v_{q2} is the parent vertex of v_1, and v_i is the parent vertex of v_{i+1} $(n > i \geq 1)$.

For the connection of another vertex, the same conclusion can be obtained by analogy. Therefore, the directions of two adjacent tracebacks are different. ■

Theorem 2 *A path S containing more than 1 vertex cannot be backtracked more than once.*

Proof *If the path S is backtracked multiple times, v_n should be a leaf when it is backtracked for the first time, which means that v_i $(1 \leq i < N)$ has been traversed by BFS, and therefore, it should be a stem. This means that the connecting vertex v_1 cannot backtrack in the second backtracking because it is a leaf. On the other hand, before v_n becomes a leaf, v_1 will first become a leaf, try to connect with v_{q1}, and backtrack to the direction of the root vertex (not passing by S).*

However, a vertex can be repeatedly calculated for the above algorithm. Assuming that the path S contains only one vertex, then the situation described in the theorem is true. ■

Theorem 3 *The time complexity of step 3 is $O(m+n)$.*

Proof *All vertices are enqueued once in this step, and its complexity is $O(n)$. The time complexity of the connection step is specifically analyzed below.*

- ***The graph G has no circles.*** *the algorithm has no connection steps, equivalent to performing BFS. So the time complexity is $O(m+n)$.*
- ***The graph G has circles, but there is no common vertex between the circles*** *(three leaf vertices at any time cannot be adjacent vertices). When the connection step appears, there is a circle in the graph G. Because the circles have no common vertex, each edge is backtracked at most once; that is, the distance is calculated at most once for each vertex. In the absorption step, there is no need to update the distance between descendants. So the time complexity is $O(m+n)$.*
- ***The graph G has circles, and there are common vertices between circles.*** *Each edge may be backtracked and absorbed multiple times. In the backtracking step, the distance must be obtained and calculated multiple times (Figure 3). In the absorption step, the distance of some vertices may be recalculated multiple times. The backtracking of adjacent circles in the connection step generates these vertices that need to be recalculated. There are the following two cases.*
  - ***The path is backtracked once and absorbed multiple times.*** *Suppose that different branches of a path are connected to multiple paths. Each connection produces backtracking and the branch is absorbed by different branches. Each branch is calculated once due to backtracking, and the distance is reset to infinity when it is absorbed for the second time. Therefore, each branch is only traversed twice.*

– ***The path is backtracked and absorbed multiple times.*** *Suppose that the same branch of a path is connected to multiple paths. Each connection produces backtracking and the branch is absorbed by other branches. According to the above theorem 1 and 2, the path can only contain a single vertex. Because each backtracking requires a corresponding new vertex, the average complexity is* $O(1)$.

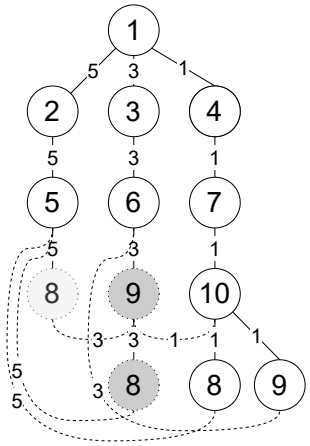

Figure 3: An Example with Multiple Abosortion (The gray vertices with different lightness shows different abosortion and the white vertices show the final state of the graph.)

Therefore, the time complexity of this algorithm is $O(m + n)$.

The space complexity of the algorithm is $O(n)$, because a hash table is needed to store the graph, and a queue is needed to traverse all vertices. In the connection step, a list is needed to store the distances of the absorbed vertices.